

Welcome back! Aapka LangChain foundation ab kaafi strong ho chuka hai. Aaj hum framework ke core engine room mein enter kar rahe hain: **Chains aur Runnables**.

Agar hum software engineering patterns ki baat karein—jaise backend mein request ek layered architecture (Controller -> Service -> Repository) se flow karti hai—toh LangChain mein yeh flow Chains aur Runnables handle karte hain.

Dhyan rahe, aaj ki class bohot special hai kyunki hum ek transition phase seekhenge: LangChain ka purana tareeqa (LLMChain) aur naya, modern tareeqa (LCEL). Let's dive in!

Part A: LLMChain (The Legacy Approach)

1. What is LLMChain? & 2. Why it was introduced

Jab LangChain start hua tha, toh unhone realise kiya ki developers baar-baar same 3 steps repeat kar rahe hain:

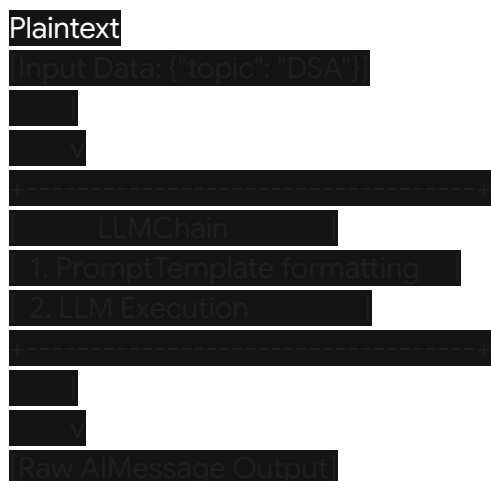
1. Template mein variable dalo.
2. Us string ko LLM ko bhejo.
3. Response wapas lo.

Is repetitive task ko ek single block mein pack karne ke liye **LLMChain** class banayi gayi. Yeh ek "wrapper" tha jo Prompt aur LLM ko ek unit mein baandh deta tha.

- **Real-life Analogy:** Socho aap kisi restaurant mein 'Fixed Thali' order karte ho. Us Thali mein Roti (Prompt) aur Sabzi (LLM) pehle se fix aur tightly packed aati hain. Aap unko alag-alag modify nahi kar sakte. LLMChain wahi Fixed Thali thi.

3. Components & 4. Flow Diagram

LLMChain ke mainly 2 hi core components hote the: PromptTemplate aur LLM.



5. Internal Working & 6. Complete Python Example

Aapke DSA tutorials ke liye ek YouTube script generator banate hain purane tareeqe se.

```
Python
import os
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain.chains import LLMChain # ⚠ Note: This is now legacy!
from langchain_core.prompts import PromptTemplate

load_dotenv()

# 1. Initialize LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

# 2. Create PromptTemplate
template = "Write a catchy 2-line intro for a Gobus Billa YouTube video about {topic}."
prompt = PromptTemplate(template=template, input_variables=["topic"])

# 3. Create the LLMChain (The old way)
legacy_chain = LLMChain(llm=llm, prompt=prompt)

# 4. Execute the chain
print("--- LLMChain Output ---")
# In older LangChain, we used .run() or passed a dictionary directly
result = legacy_chain.invoke({"topic": "Binary Search Trees"})
print(result["text"]) # Output was often hidden inside a dictionary key
```

Dry Run: Line 3 mein humne legacy LLMChain import kiya. Line 17 mein humne ek object banaya jisme dono prompt aur llm pass kiye. Jab humne .invoke() call kiya, LLMChain ne internal validation ki, prompt format kiya aur LLM se response laakar ek dictionary {'topic': '...', 'text': '...'} mein de diya.

Common Bugs in LLMChain:

- Output format unpredictable hota tha (kabhi string aati thi, kabhi dict).
- Isme Custom Parsers ya naye steps add karna (jaise translation) bohot complicated code maangta tha.

Part B: RunnableSequence & LCEL (The Modern Approach)

1. Why move to LCEL? & 2. What are Runnables?

Purana LLMChain ek monolithic design tha. Jaise-jaise GenAI complex hua (streaming chahiye, async processing chahiye, output parsing chahiye), LLMChain break hone laga.

LangChain ne **LCEL (LangChain Expression Language)** introduce kiya. Yeh Unix/Linux terminal pipes (|) ya Java Streams (.map().filter()) se inspired hai.

LCEL mein har component (Prompt, LLM, Parser) ek **Runnable** interface implement karta hai.

Runnable ka matlab hai "koi aisi cheez jisko .invoke() kiya ja sake".

3. RunnableSequence & 4. The Pipe | Operator

Jab aap multiple Runnables ko pipe | operator se jodte hain, toh wo automatically ek **RunnableSequence** ban jata hai. Left side ka output instantly right side ka input ban jata hai.

- **Real-life Analogy:** Yeh ek modern Buffet system ya custom assembly line jaisa hai. Plate uthao (Input) -> Chawal dalo (Prompt) -> Dal dalo (LLM) -> Salad dalo (Parser). Aap kisi bhi step ko easily replace ya add kar sakte ho bina doosre steps ko tode.

5. Flow Diagram of LCEL



5. & 6. Complete Python Example (Modern Way)

```

Python
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser

# 1. Initialize components
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)
parser = StrOutputParser()

template = "Write a catchy 2-line intro for a Gobus Billa YouTube video about {topic}."
prompt = PromptTemplate.from_template(template)

# 2. Build the RunnableSequence using LCEL
modern_chain = prompt | llm | parser

# 3. Execute
print("\n--- LCEL / RunnableSequence Output ---")
# .invoke() takes the dictionary and passes it sequentially through the pipe
result = modern_chain.invoke({"topic": "Linked Lists vs Arrays"})
print(result)

```

Dry Run:

Yahan prompt | llm | parser ek chain of events banata hai. invoke() call hote hi {"topic": "Linked Lists vs Arrays"} prompt ke paas jata hai. Prompt usey string banata hai aur | ke through llm ko push karta hai. llm ka response AI Message ke form mein parser ke paas jata hai. parser finally sirf text nikal kar humein de deta hai.

7. Difference Between Old and New Approach

Feature	LLMChain (Legacy)	RunnableSequence (LCEL)
Syntax	chain = LLMChain(llm=llm, prompt=prompt)	`chain = prompt
Modularity	Tightly coupled (Monolith)	Highly decoupled (Microservices-like)

Streaming	Difficult to implement	Natively supported via <code>.stream()</code>
Flexibility	Hard to add more than 2 steps	Infinite chaining (`A

8. Production Best Practices

- **Always use LCEL:** Naye projects mein kabhi bhi LLMChain import mat karo.
- **Keep chains small:** Ek lambi 10-step ki chain banane se behtar hai choti-choti sub-chains banayein aur unhe compose karein.
- **Logging:** LCEL natively LangSmith ko support karta hai. Production mein API costs track karne ke liye apni chains ko properly trace karein.

Review Section

Interview Questions & Answers

- **Q: What does the pipe | operator actually do behind the scenes in LangChain?**
 - *Answer:* Behind the scenes, LangChain overloads the `__or__` dunder method in Python for all classes that inherit from Runnable. It takes the output of the left operand and pipes it as the `.invoke()` input for the right operand, creating a RunnableSequence.
- **Q: Why is StrOutputParser almost always necessary in LCEL, but wasn't in LLMChain?**
 - *Answer:* LLMChain internally extracted the text from the model's response. In LCEL, components are decoupled and un-opinionated. The LLM returns a rich AIMessage object (with metadata, token usage, etc.). The parser is needed to explicitly extract the string content.

Exercises & Mini Assignment

- **Task 1:** Write a 3-step LCEL chain: The prompt asks for a recipe, the LLM generates it, and a custom parser (just use Python string methods) converts the output to UPPERCASE.
- **Task 2:** Create two separate chains. Chain A generates a video title. Chain B takes a title and generates a thumbnail description. Connect them: `chain_a | chain_b`.